C. Permutation Operations

time limit per test 2 seconds
memory limit per test 256 megabytes
input standard input
output standard output

Rating: 1300 | Tags: constructive algorithms, greedy, implementation, math | https://codeforces.com/problemset/problem/1746/C

You are given a permutation a of size n and you should perform n operations on it. In the i-th operation, you can choose a non-empty suffix of a and increase all of its elements by i. How can we perform the operations to minimize the number of inversions in the final array?

Note that you can perform operations on the same suffix any number of times you want.

A permutation of size n is an array of size n such that each integer from 1 to n occurs exactly once in this array. A suffix is several consecutive elements of an array that include the last element of the array. An inversion in an array a is a pair of indices (i, j) such that i gt j and a_i lt a_j.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \le t \le 10^4$). The description of the test cases follows.

The first line of each test case contains a single integer n ($1 \le n \le 10^5$) — the size of the array.

The second line contains n distinct integers $a_1, a_2, \ldots, a_n$ ($1 \le a_i \le n$), the initial permutation a.

It's guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, print n integers $x_1, x_2, \ldots, x_n$ ($1 \le x_i \le n$ for each $1 \le i \le n$) indicating that the i-th operation must be applied to the suffix **starting at index** x_i. If there are multiple answers, print any of them.

Example

Input

```
4
4
1 2 3 4
5
1 3 2 4 5
3
2 3 1
1
1
```

Output

```
1 1 1 1
1 4 3 2 1
1 3 3
1
```

Note

In the first test case one of the optimal solutions is to increase the whole array on each operation (that is, choose the suffix starting at index 1). The final array [11, 12, 13, 14] contains 0 inversions.

In the second test case, *a* will be equal to [2, 4, 3, 5, 6], [2, 4, 3, 7, 8], [2, 4, 6, 10, 11], [2, 8, 10, 14, 15] and [7, 13, 15, 19, 20] after the first, second, third, fourth, and fifth operations, respectively. So the final array *a* has zero inversions.